BONOBO - Documento Maestro del Proyecto

1. Introducción.... 3
1.1. Propósito de este Documento.... 3
1.2. Descripción del Proyecto.... 3
1.3. Objetivos del Proyecto.... 3
1.3.1. Objetivos del Producto.... 3
1.3.2. Objetivos Académicos.... 3
1.4. Alcance.... 4
1.4.1. Incluido.... 4
1.4.2. No Incluido.... 4
2. Visión del Producto.... 4
2.1. Problema.... 4
2.2. Propuesta de Valor.... 4
2.3. Usuario Objetivo.... 5
3. Historias de Usuario.... 5
Módulo de Autenticación (Nivel 1).... 5
Módulo de Películas (Nivel 1).... 5
Módulo de Perfil (Nivel 1).... 6
Módulo Social (Nivel 2).... 6
Módulo de Listas Personalizadas (Nivel 2).... 7
Módulo de Gamificación (Nivel 3).... 7
4. Escenarios de Usuario.... 8
Escenario 1: Álex acaba de ver una película y quiere registrarla rápido (Nivel 1).... 8
Escenario 2: Álex quiere escribir una reseña detallada (Nivel 1).... 8
Escenario 3: Álex organiza sus películas (Nivel 1).... 8
Escenario 4: Álex quiere saber qué opinan sus amigos de una película (Nivel 2).... 9
Escenario 5: Álex crea una lista personalizada (Nivel 2).... 9
Escenario 6: Álex desbloquea un logro (Nivel 3).... 9
5. Requisitos del Sistema.... 9
5.1. Requisitos Funcionales.... 9
Nivel 1 — MVP.... 10
Nivel 2 — Social.... 11
Nivel 3 — Gamificación.... 12
5.2. Requisitos No Funcionales.... 12
6. Casos de Uso.... 13
6.1. Actores del Sistema.... 13
6.2. Diagrama de Casos de Uso.... 14
6.3. Descripción Detallada de los Casos de Uso.... 14
CU-01: Registrarse.... 14
CU-02: Iniciar sesión.... 15
CU-03: Cerrar sesión.... 15
CU-04: Buscar película.... 15
CU-05: Ver detalle de película.... 16

CU-06: Marcar como vista....16
CU-07: Puntuar película....16
CU-08: Registrar en diario....17
CU-09: Seguir a un usuario....17
CU-10: Ver reseñas de amigos en una película....17
CU-11: Comentar en una reseña....18
CU-12: Crear y gestionar una lista personalizada....18
CU-13: Reclamar XP de un logro....18
7. Hoja de Ruta del Producto....19
Nivel 1 — MVP (Producto Mínimo Viable)....19
Nivel 2 — Social + Listas....19
Nivel 3 — Gamificación....20
Resumen visual....20
8. Arquitectura del Sistema....20
8.1. Descripción General....20
8.2. Diagrama de Arquitectura....21
8.3. Flujo de una Petición Típica....21
9. Modelo de Datos....22
9.1. Modelo Conceptual....22
9.2. Relaciones entre Entidades....23
9.3. Diagrama Entidad-Relación....24
9.4. Tablas de la Base de Datos....24
Nivel 1 — MVP....24
Nivel 2 — Social....26
Nivel 3 — Gamificación....28
10. Diseño de la API....29
Autenticación....29
Películas....29
Acciones sobre Películas....29
Perfil y Usuarios....30
Social....31
Diario Expandido....31
Listas Personalizadas....32
Gamificación....32
11. Paquete Tecnológico....33
Equipo....33

1. Introducción

1.1. Propósito de este Documento

Este documento sirve como referencia central del proyecto Bonobo. Su objetivo es reunir en un solo lugar toda la planificación, diseño y decisiones técnicas del proyecto, de forma que cualquier miembro del equipo pueda consultarlo para entender qué estamos construyendo, por qué y cómo.

El documento cubre desde la visión del producto y los requisitos hasta el diseño técnico completo (modelo de datos, endpoints, arquitectura). Está pensado para mantenerse actualizado durante el desarrollo y servir también como base para la memoria del TFG.

1.2. Descripción del Proyecto

Bonobo es una aplicación web para registrar y valorar películas con un sistema de gamificación integrado. Funciona como un diario cinematográfico donde el usuario puede buscar películas, marcarlas como vistas, puntuarlas, escribir reseñas, organizar listas y llevar un seguimiento de su actividad. A medida que interactúa con la aplicación, desbloquea logros y sube de nivel.

En una frase: **registrar películas es el medio, el juego es el fin.**

La aplicación se inspira en plataformas como Letterboxd, pero se diferencia en dos aspectos clave: prioriza la rapidez de uso (registrar una película en pocos clics) y añade una capa de gamificación (logros, experiencia y niveles) que convierte el registro de películas en una experiencia más motivante y divertida.

1.3. Objetivos del Proyecto

1.3.1. Objetivos del Producto

- Desarrollar una aplicación web funcional donde los usuarios puedan registrarse, buscar películas y guardar un registro completo de su actividad cinematográfica.
- Diseñar una interfaz que priorice la rapidez: registrar una película en menos de 3 clics.
- Implementar un componente social que permita seguir a otros usuarios, ver sus reseñas y comentar en ellas.
- Implementar un sistema de logros, experiencia y niveles que incentive el uso continuado de la aplicación.

1.3.2. Objetivos Académicos

- Separar claramente el backend (API REST con FastAPI) del frontend (HTML/CSS/JS), permitiendo que ambos se desarrollen de forma independiente.
- Diseñar e implementar una base de datos relacional PostgreSQL con múltiples tablas y relaciones.
- Consumir una API externa (TMDB) para obtener datos de películas en tiempo real.
- Utilizar control de versiones (Git/GitHub) de forma colaborativa durante todo el desarrollo.
- Entregar documentación técnica completa: diagramas, especificación de endpoints, manual de despliegue.
- Desplegar la aplicación en un servidor real (VM Ubuntu en la nube).

1.4. Alcance

1.4.1. Incluido

- Sistema completo de autenticación (registro, login, logout).
- Búsqueda de películas por título mediante la API de TMDB.
- Registro de actividad cinematográfica: marcar como vista, puntuar, escribir reseñas en un diario, marcar favoritas, crear una watchlist y dar me gusta.
- Listas personalizadas de películas (públicas y privadas).
- Perfil de usuario público con toda su actividad y personalización (foto, bio).
- Sistema social: seguir usuarios, ver reseñas de amigos y de otros usuarios en las películas, comentar y dar like a reseñas.
- Sistema de gamificación: logros desbloqueables, puntos de experiencia (XP) y niveles.
- Despliegue en una VM en la nube (Oracle Cloud o AWS).

1.4.2. No Incluido

- Aplicación móvil nativa (solo web).
- Chat en tiempo real entre usuarios.
- Grupos o comunidades.
- Contenido de series de televisión (solo películas).
- Sistema de recomendación automática de películas.
- Moderación de contenido automatizada.
- Eliminación de cuenta de usuario.
- Monetización o pagos.
- Eliminación de entradas del diario (el diario es un registro histórico inmutable en cuanto a su existencia, pero el contenido de una entrada — reseña y fecha — sí es editable).

2. Visión del Producto

2.1. Problema

Las plataformas actuales de registro de películas como Letterboxd están orientadas a un público cinéfilo que disfruta escribiendo reseñas elaboradas. Sin embargo, la mayoría de los espectadores no son críticos de cine: simplemente quieren apuntar lo que han visto, darle una nota rápida y quizás ver qué opinan sus amigos. Para este perfil de usuario, Letterboxd resulta excesivo, y las alternativas más simples carecen de una razón para volver a la app una vez registrada la película.

2.2. Propuesta de Valor

Bonobo ofrece una experiencia de registro de películas minimalista y rápida combinada con un sistema de gamificación que da al usuario una razón para seguir usando la app. La propuesta se resume en tres pilares:

- **Rapidez:** Registrar una película en menos de 3 clics. Sin fricciones, sin obligar a escribir reseñas largas.
- **Social:** Ver qué ven tus amigos, leer sus reseñas y comentar en ellas. La actividad cinematográfica como actividad compartida.

- **Progresión:** Desbloquear logros, ganar experiencia y subir de nivel convierte el registro de películas en un juego. "Registrar películas es el medio, el juego es el fin."

2.3. Usuario Objetivo

Nombre: Álex, 22 años, estudiante universitario.

Comportamiento: Ve entre 8 películas al mes, normalmente los fines de semana o en plataformas de streaming. Suele pedir recomendaciones a sus amigos y compartir opiniones sobre lo que ha visto. Le gustan los videojuegos y responde bien a sistemas de logros y progreso: le motiva desbloquear cosas.

Qué necesita de Bonobo: Una app rápida. Solo quiere buscar "Dune", darle un 8 sobre 10, marcarla como vista y ver qué puntuación le han dado sus amigos. Si le apetece, escribe una reseña corta. Si además desbloquea un logro por haberla visto, mejor.

Qué NO necesita: Escribir una reseña de 500 palabras, navegar por menús complicados, o configurar opciones avanzadas.

Frase que lo resume: "Quiero registrar pelis tan rápido como doy like en Instagram."

3. Historias de Usuario

Las historias de usuario describen qué quiere hacer el usuario y por qué. Utilizan el formato: **"Como [tipo de usuario], quiero [acción] para [beneficio]."** Cada historia está asignada al nivel en el que se implementará.

Módulo de Autenticación (Nivel 1)

ID	Historia	Prioridad
HU-01	Como visitante, quiero registrarme con email, username y contraseña para crear mi cuenta en Bonobo.	Alta
HU-02	Como usuario registrado, quiero iniciar sesión para acceder a mi perfil y mis datos.	Alta
HU-03	Como usuario autenticado, quiero cerrar sesión para proteger mi cuenta.	Alta

Módulo de Películas (Nivel 1)

ID	Historia	Prioridad
HU-04	Como usuario, quiero ver películas en cartelera y próximos estrenos en la pantalla de inicio para descubrir qué hay disponible.	Media
HU-05	Como usuario, quiero buscar películas por título para encontrar una película concreta rápidamente.	Alta

HU-06	Como usuario, quiero ver el detalle de una película (póster, sinopsis, reparto, géneros) para decidir si me interesa.	Alta
HU-07	Como usuario, quiero marcar una película como vista con un solo clic para registrarla rápidamente sin tener que añadir más datos. También quiero poder desmarcarla si no tengo datos asociados (diario, puntuación o me gusta).	Alta
HU-08	Como usuario, quiero puntuar una película del 1 al 10 para dejar constancia de mi opinión. También quiero poder quitar mi puntuación si me arrepiento.	Alta
HU-09	Como usuario, quiero registrar una película en mi diario con fecha de visionado y reseña opcional para llevar un registro detallado de cuándo la vi y qué me pareció. También quiero poder editar la reseña o la fecha de visionado de una entrada existente si me he equivocado.	Media
HU-10	Como usuario, quiero dar "me gusta" a una película para marcarla como especial sin necesidad de puntuarla.	Baja
HU-11	Como usuario, quiero añadir películas a mi watchlist para recordar qué quiero ver en el futuro.	Alta
HU-12	Como usuario, quiero configurar hasta 4 películas favoritas desde mi perfil para mostrarlas destacadas como escaparate de mis gustos.	Media

Módulo de Perfil (Nivel 1)

ID	Historia	Prioridad
HU-13	Como usuario, quiero ver mi perfil con mi foto, bio, películas favoritas, películas vistas, diario y watchlist para tener una visión completa de mi actividad.	Alta
HU-14	Como usuario, quiero editar mi perfil (foto y bio) para personalizarlo.	Media
HU-15	Como usuario, quiero ver el perfil de otros usuarios para descubrir qué ven y qué les gusta.	Media
HU-16	Como usuario, quiero buscar otros usuarios por username para encontrar a mis amigos.	Media

Módulo Social (Nivel 2)

ID	Historia	Prioridad
HU-17	Como usuario, quiero seguir a otros usuarios para tenerlos como amigos y ver su actividad.	Alta

HU-18	Como usuario, quiero dejar de seguir a un usuario si ya no me interesa su actividad.	Alta
HU-19	Como usuario, quiero ver una sección de reseñas de mis amigos en el detalle de una película para saber qué opinan las personas que sigo.	Alta
HU-20	Como usuario, quiero ver una sección general de reseñas de todos los usuarios en el detalle de una película para descubrir opiniones diversas.	Media
HU-21	Como usuario, al ver la reseña de un amigo en una película, quiero pulsar y ver todas sus entradas de diario para esa película si tiene varias.	Media
HU-22	Como usuario, quiero comentar en la reseña de otro usuario para compartir mi opinión sobre su reseña.	Media
HU-23	Como usuario, quiero dar like a la reseña de otro usuario para mostrar que estoy de acuerdo o que me gusta lo que ha escrito.	Media
HU-24	Como usuario, quiero ver cuántos seguidores tengo y a cuántas personas sigo en mi perfil.	Baja
HU-25	Como usuario, quiero ver la lista de seguidores y seguidos de cualquier usuario.	Baja

Módulo de Listas Personalizadas (Nivel 2)

ID	Historia	Prioridad
HU-26	Como usuario, quiero crear listas personalizadas de películas (ej: "Terror favorito", "Para ver con amigos") para organizar mis películas como yo quiera.	Media
HU-27	Como usuario, quiero añadir y quitar películas de mis listas personalizadas.	Media
HU-28	Como usuario, quiero editar el nombre y la descripción de mis listas.	Baja
HU-29	Como usuario, quiero eliminar una lista que ya no me interese.	Baja
HU-30	Como usuario, quiero marcar mis listas como públicas o privadas para controlar quién puede verlas.	Media
HU-31	Como visitante o usuario, quiero ver las listas públicas de otros usuarios desde su perfil para descubrir sus recopilaciones.	Media

Módulo de Gamificación (Nivel 3)

ID	Historia	Prioridad
HU-32	Como usuario, quiero desbloquear logros al completar ciertas acciones (primera reseña, 10 películas vistas, etc.) para sentir que progreso.	Alta
HU-33	Como usuario, quiero reclamar la XP de mis logros desbloqueados para subir de nivel.	Alta
HU-34	Como usuario, quiero ver mi nivel, XP total y logros en mi perfil para saber cuánto he progresado.	Alta
HU-35	Como usuario, quiero ver un ranking de nivel entre las personas que sigo para competir amistosamente.	Media
HU-36	Como usuario, quiero ver los logros disponibles y cuáles me faltan por desbloquear para saber qué me queda por hacer.	Media

4. Escenarios de Usuario

Los escenarios describen flujos concretos paso a paso, mostrando cómo interactúa el usuario con la aplicación en situaciones reales.

Escenario 1: Álex acaba de ver una película y quiere registrarla rápido (Nivel 1)

1. Álex abre Bonobo en su navegador (ya tiene sesión iniciada).
2. Pulsa en la barra de búsqueda y escribe "Dune".
3. Aparecen los resultados de TMDB. Pulsa en "Dune: Part Two (2024)".
4. Se abre la ficha de la película con póster, sinopsis, reparto y géneros.
5. Álex pulsa el botón "Vista". La película queda registrada como vista instantáneamente.
6. Arrastra el selector de puntuación al 8. Se guarda automáticamente.
7. Álex vuelve a la pantalla de inicio. Tiempo total: unos 15 segundos.

Escenario 2: Álex quiere escribir una reseña detallada (Nivel 1)

1. Álex busca "Parásitos" y abre su ficha.
2. En lugar de pulsar solo "Vista", pulsa "Añadir al diario".
3. Se abre un formulario con: fecha de visionado (por defecto hoy) y campo de reseña.
4. Álex escribe: "La vi con mi hermano, nos voló la cabeza el giro del sótano."
5. Pulsa guardar. La película queda marcada como vista automáticamente.
6. La entrada aparece en su diario con la fecha y la reseña.

Escenario 3: Álex organiza sus películas (Nivel 1)

1. Álex ve un tráiler de una película que le interesa y abre Bonobo para no olvidarla.

2. Busca la película y pulsa el icono de watchlist (bookmark). La película se añade a su lista de "pendientes".
3. Más tarde, navega por la cartelera en la pantalla de inicio y encuentra una película que le encantó hace tiempo. Abre su ficha y le da a "Me gusta" (corazón).
4. Álex quiere destacar sus películas favoritas en su perfil. Va a "Editar perfil" y en la sección "Películas favoritas" ve 4 huecos.
5. Busca "Parásitos" y la coloca en el hueco 1. Luego añade "Dune", "Blade Runner 2049" y "Interstellar" en los huecos 2, 3 y 4.
6. Guarda. Ahora cualquier persona que visite su perfil ve esas 4 películas destacadas en la cabecera.

Escenario 4: Álex quiere saber qué opinan sus amigos de una película (Nivel 2)

1. Álex abre la ficha de "Oppenheimer".
2. Debajo de los datos de la película, ve una sección "Tus amigos" con las entradas de sus amigos: avatares, nombres y su última reseña si la tienen.
3. Ve que su amigo Mario tiene una reseña. Pulsa sobre ella.
4. Como Mario ha visto Oppenheimer dos veces y tiene dos entradas en el diario, Álex ve ambas reseñas con sus fechas.
5. Álex comenta en la segunda reseña de Mario: "Tienes razón, la escena del juicio es brutal."
6. También le da like a la reseña.
7. Más abajo, en la sección "Reseñas generales", Álex puede leer reseñas de usuarios que no sigue.

Escenario 5: Álex crea una lista personalizada (Nivel 2)

1. Álex va a su perfil y pulsa "Crear nueva lista".
2. Le pone nombre: "Ciencia ficción imprescindible" y descripción: "Las mejores pelis de sci-fi que he visto".
3. La marca como pública para que otros usuarios puedan verla.
4. Busca "Blade Runner 2049" y desde la ficha de la película pulsa "Añadir a lista" y selecciona "Ciencia ficción imprescindible".
5. Repite con "Arrival", "Interstellar" y "Dune".
6. Mario visita el perfil de Álex y ve la lista "Ciencia ficción imprescindible" entre sus listas públicas. Pulsa y ve las 4 películas.
7. Más tarde, Álex decide crear otra lista "Guilty pleasures" y la marca como privada. Solo él puede verla.

Escenario 6: Álex desbloquea un logro (Nivel 3)

1. Álex acaba de registrar su décima película del mes.
2. Aparece una notificación: "¡Logro desbloqueado: Maratonista! (Registrar 10 películas en un mes)".
3. Álex va a su perfil, sección de logros, y ve el logro nuevo con un botón "Reclamar XP".
4. Pulsa reclamar. Se le suman 50 XP a su total.
5. Su barra de XP avanza y ve que le faltan 20 XP para subir al nivel 5.
6. Consulta el ranking de amigos y ve que ha adelantado a su amigo Bladi.

5. Requisitos del Sistema

5.1. Requisitos Funcionales

Los requisitos funcionales describen **qué** debe hacer el sistema. Se derivan de las historias de usuario y se agrupan por nivel.

Nivel 1 — MVP

ID	Requisito	HU
RF-01	El sistema debe permitir el registro de usuarios con email, username y contraseña. La contraseña se almacena como hash bcrypt.	HU-01
RF-02	El sistema debe autenticar usuarios mediante JWT y devolver un token de acceso válido por 24 horas.	HU-02
RF-03	El sistema debe permitir cerrar sesión invalidando el token en el lado del cliente.	HU-03
RF-04	El sistema debe mostrar películas en cartelera y próximos estrenos consumiendo la API de TMDB. Accesible sin autenticación.	HU-04
RF-05	El sistema debe permitir buscar películas por título a través de TMDB y devolver resultados paginados. Accesible sin autenticación.	HU-05
RF-06	El sistema debe mostrar el detalle de una película (título, póster, sinopsis, reparto, géneros) obtenido de TMDB en tiempo real. Accesible sin autenticación. Si el usuario envía un token JWT válido, la respuesta se enriquece con su estado personal con la película (vista, puntuación, me gusta, watchlist).	HU-06
RF-07	El sistema debe permitir marcar una película como vista. Se puede desmarcar solo si el usuario no tiene entradas de diario, puntuación ni me gusta para esa película. El frontend desactiva el botón de desmarcar si no se cumplen las condiciones y muestra un indicador explicativo. Al marcar como vista, si la película estaba en la watchlist del usuario, se elimina automáticamente. Las favoritas y la watchlist no bloquean el desmarcado.	HU-07
RF-08	El sistema debe permitir puntuar una película del 1 al 10 (enteros). La puntuación se almacena en la tabla Vista y se actualiza siempre con la última puntuación dada. Si no existe un registro de Vista para ese usuario-película, se crea automáticamente. Si no existe Vista y se crea automáticamente, y la película estaba en la watchlist, se elimina automáticamente.El usuario puede eliminar su puntuación en cualquier momento (vuelve a null). La Vista se mantiene. Esto se realiza mediante un endpoint dedicado (DELETE puntuación).	HU-08

RF-09	El sistema debe permitir crear entradas en el diario con fecha de visionado y reseña opcional. Al crear una entrada de diario, la película se marca como vista automáticamente si no lo estaba ya. Una película puede tener múltiples entradas de diario del mismo usuario. Si la película estaba en la watchlist del usuario, se elimina automáticamente de la watchlist. El usuario puede editar la reseña y la fecha de visionado de una entrada existente. No puede eliminar entradas.	HU-09
RF-10	El sistema debe permitir dar/quitar "me gusta" a una película (toggle).	HU-10
RF-11	El sistema debe permitir añadir/quitar películas de la watchlist del usuario (toggle).	HU-11
RF-12	El sistema debe permitir configurar hasta 4 películas favoritas desde la sección de edición del perfil. El usuario busca películas y las asigna a sus huecos de favoritas. Si ya tiene 4 y quiere cambiar una, debe quitar una primero. Las favoritas se muestran destacadas en la cabecera del perfil. No es necesario que la película esté marcada como vista para añadirla como favorita.	HU-12
RF-13	El sistema debe mostrar el perfil de usuario con: foto, bio, películas favoritas, lista de películas vistas con puntuación, diario de visionado y watchlist.	HU-13
RF-14	El sistema debe permitir editar el perfil: foto de perfil (URL), biografía y películas favoritas (hasta 4).	HU-14, HU-12
RF-15	El sistema debe mostrar el perfil público de cualquier usuario con la misma información. Accesible sin autenticación.	HU-15
RF-16	El sistema debe permitir buscar usuarios por username con resultados paginados. Accesible sin autenticación.	HU-16
RF-17	Todos los endpoints que devuelven listas deben aceptar parámetros de paginación: skip (defecto 0) y limit (defecto 20).	Transversal
RF-18	Todos los endpoints deben devolver errores con HTTPException incluyendo un campo "detail" descriptivo y códigos HTTP estándar (400, 401, 404, 422).	Transversal
RF-19	Al interactuar con una película por primera vez (vista, puntuación, diario, me gusta, watchlist, o al añadirla como favorita desde el perfil), el sistema debe crear un registro en la tabla Pelicula con los datos de caché local (tmdb_id, título, póster, año) si no existe ya.	Transversal

Nivel 2 — Social

ID	Requisito	HU

RF-20	El sistema debe permitir seguir/dejar de seguir a otro usuario.	HU-17, HU-18
RF-21	En el detalle de una película, el sistema debe mostrar una sección con las entradas de diario (reseñas) de los usuarios que el usuario actual sigue. Se muestra una entrada por amigo: al pulsar, se ven todas sus entradas de diario para esa película. Si el amigo solo marcó la película como vista sin reseña, se indica pero no hay enlace.	HU-19, HU-21
RF-22	En el detalle de una película, el sistema debe mostrar una sección general con las reseñas de todos los usuarios, paginadas.	HU-20
RF-23	El sistema debe permitir escribir un comentario en una entrada de diario de cualquier usuario. No se permiten respuestas a comentarios.El autor de un comentario puede eliminar su propio comentario. No se pueden eliminar comentarios de otros usuarios.	HU-22
RF-24	El sistema debe permitir dar/quitar like a una entrada de diario de cualquier usuario (toggle).	HU-23
RF-25	El perfil de usuario debe mostrar su número de seguidores y seguidos.	HU-24
RF-26	El sistema debe permitir ver la lista de seguidores y seguidos de cualquier usuario, con paginación.	HU-25
RF-27	El sistema debe permitir crear listas personalizadas de películas con nombre y descripción.	HU-26
RF-28	El sistema debe permitir añadir/quitar películas de una lista personalizada propia.	HU-27
RF-29	El sistema debe permitir editar el nombre y la descripción de una lista propia.	HU-28
RF-30	El sistema debe permitir eliminar una lista propia (se eliminan las asociaciones lista-película, no las películas).	HU-29
RF-31	El sistema debe permitir marcar una lista como pública o privada. Las listas privadas solo son visibles para su creador. Las públicas son visibles para cualquier visitante o usuario.	HU-30
RF-32	El perfil de usuario debe mostrar sus listas públicas. Si el usuario ve su propio perfil, ve también las privadas.	HU-31
RF-33	El sistema debe permitir ver el contenido (películas) de una lista pública de cualquier usuario, con paginación.	HU-31

Nivel 3 — Gamificación

ID	Requisito	HU
RF-34	El sistema debe verificar automáticamente si el usuario cumple las condiciones de un logro tras cada acción relevante. Si las cumple, se registra el logro como desbloqueado.	HU-32
RF-35	El sistema debe permitir al usuario reclamar la XP de un logro desbloqueado, sumándola a su XP total y recalculando su nivel (nivel = XP_total / 100, redondeado hacia abajo, mínimo 1).	HU-33
RF-36	El perfil de usuario debe mostrar su nivel, XP total y lista de logros desbloqueados (con fecha).	HU-34
RF-37	El sistema debe mostrar un ranking de nivel entre los usuarios que el usuario actual sigue, ordenado de mayor a menor nivel.	HU-35
RF-38	El sistema debe mostrar la lista completa de logros disponibles, indicando cuáles están desbloqueados y cuáles no.	HU-36

5.2. Requisitos No Funcionales

Los requisitos no funcionales describen **cómo** debe comportarse el sistema.

ID	Requisito	Categoría
RNF-01	Las contraseñas deben almacenarse como hash bcrypt, nunca en texto plano.	Seguridad
RNF-02	Todas las rutas que modifican datos (POST, PUT, DELETE) deben requerir autenticación JWT válida. Las rutas de solo lectura (GET) de contenido público son accesibles sin autenticación.	Seguridad
RNF-03	Las puntuaciones deben validarse en el servidor (enteros entre 1 y 10). No se debe confiar solo en la validación del frontend.	Seguridad
RNF-04	La API debe responder en menos de 2 segundos para operaciones que no dependan de TMDB.	Rendimiento
RNF-05	La tabla Pelicula debe mantener un caché local (título, póster, año) para no depender de TMDB al mostrar listados.	Rendimiento
RNF-06	La interfaz debe ser funcional en navegadores modernos (Chrome, Firefox, Safari, Edge).	Compatibilidad
RNF-07	El código debe estar versionado en GitHub con commits descriptivos y ramas si es necesario.	Mantenibilidad

RNF-08	El backend y el frontend deben estar completamente separados, comunicándose exclusivamente mediante API REST (JSON).	Arquitectura
RNF-09	La aplicación debe poder desplegarse en una VM Ubuntu en Oracle Cloud o AWS.	Despliegue
RNF-10	El sistema debe manejar errores de TMDB con gracia: si TMDB no responde, mostrar datos cacheados cuando sea posible o un mensaje de error claro cuando no lo sea.	Resiliencia
RNF-11	Ningún endpoint debe devolver más de 20 resultados por defecto sin paginación.	Rendimiento
RNF-12	Las contraseñas deben tener una longitud mínima de 8 caracteres. Se valida en el servidor.	Seguridad
RNF-13	El formato del email debe validarse en el servidor antes de crear la cuenta.	Seguridad
RNF-14	Los campos de texto introducidos por el usuario (reseñas, comentarios, bio, nombres de listas) deben sanitizarse para prevenir inyección de código (XSS).	Seguridad
RNF-15	La interfaz debe ser usable en dispositivos móviles (diseño responsive).	Usabilidad

6. Casos de Uso

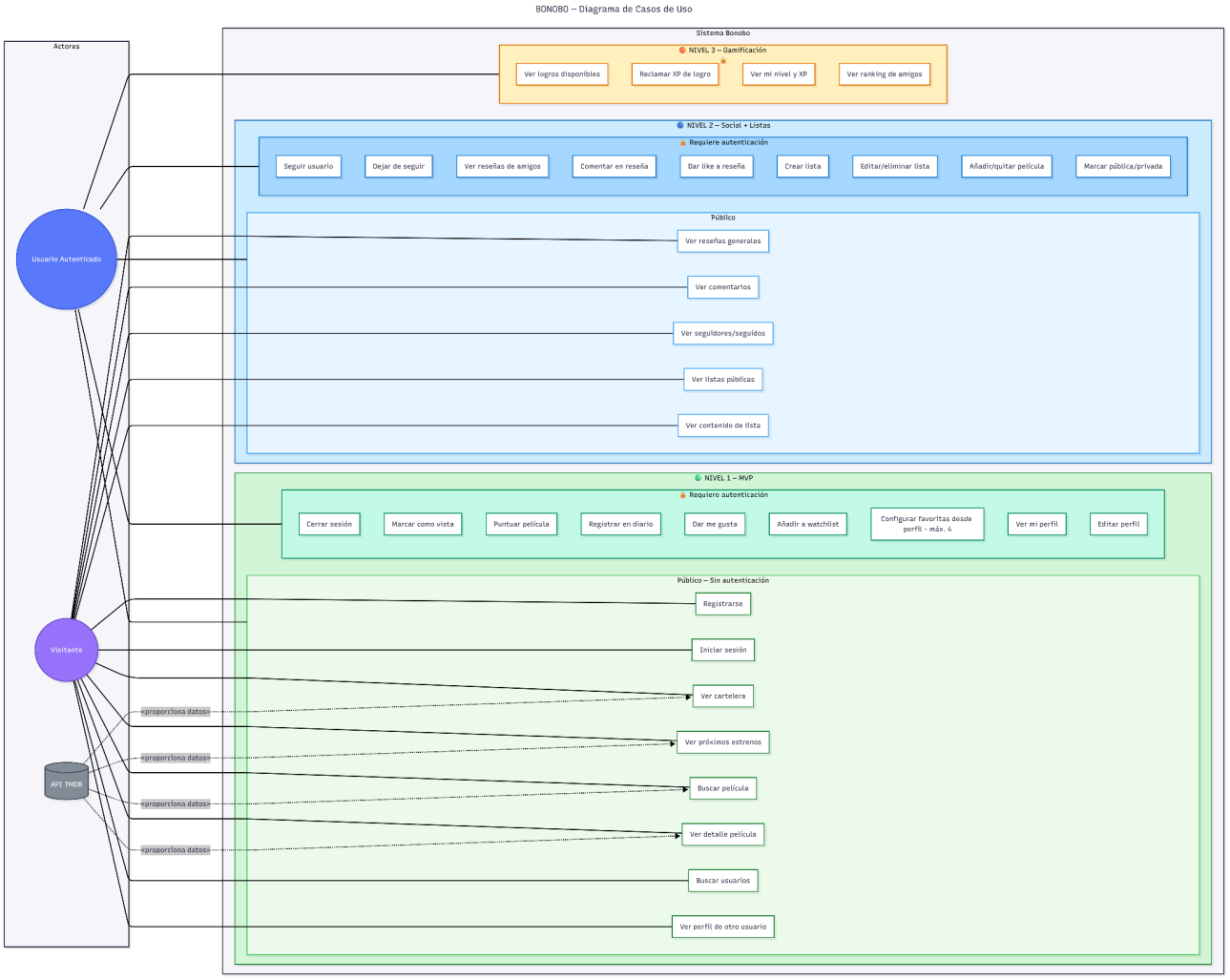
6.1. Actores del Sistema

Visitante: Persona que accede a Bonobo sin estar autenticada. Puede navegar libremente por la aplicación: ver la cartelera, buscar películas, consultar el detalle de cualquier película (incluyendo las reseñas de otros usuarios), buscar usuarios y ver sus perfiles públicos. Lo que NO puede hacer es interactuar: no puede marcar películas como vistas, puntuar, escribir reseñas, dar me gusta, seguir usuarios, comentar ni ninguna acción que requiera cuenta. Puede registrarse e iniciar sesión.

Usuario autenticado: Persona que ha iniciado sesión en Bonobo. Puede hacer todo lo que hace un visitante y además realizar todas las acciones de interacción: marcar películas como vistas, puntuar, registrar en el diario, dar me gusta, gestionar su watchlist y favoritas, editar su perfil, seguir usuarios, comentar en reseñas y progresar con logros.

API de TMDB (sistema externo): Servicio externo que proporciona los datos de películas. El backend consulta TMDB para: búsquedas por título (`/search/movie`), cartelera actual (`/movie/now_playing`), próximos estrenos (`/movie/upcoming`), detalle de película (`/movie/{id}`) y reparto (`/movie/{id}/credits`). Bonobo no almacena estos datos completos, sino que los consulta en tiempo real y guarda solo un caché mínimo (título, póster, año).

6.2. Diagrama de Casos de Uso



6.3. Descripción Detallada de los Casos de Uso

CU-01: Registrarse

- **Actor principal:** Visitante
- **Precondición:** El visitante no tiene una cuenta en Bonobo.
- **Flujo principal:**
    1. El visitante accede a la pantalla de registro.
    2. Introduce email, username y contraseña.
    3. El sistema valida que el email y el username no están ya registrados.
    4. El sistema crea la cuenta, almacenando la contraseña como hash bcrypt.
    5. El sistema redirige al visitante a la pantalla de login.
- **Flujo alternativo (email o username ya existe):** 3a. El sistema detecta que el email o username ya existe. 3b. El sistema devuelve un error 400 con detalle: "El email ya está registrado" o "El username ya está en uso".
- **Postcondición:** Existe un nuevo usuario en la base de datos.
- **Nivel:** 1 (MVP)

CU-02: Iniciar sesión

- **Actor principal:** Visitante

- **Precondición:** El visitante tiene una cuenta registrada.
- **Flujo principal:**
    1. El visitante accede a la pantalla de login.
    2. Introduce email y contraseña.
    3. El sistema verifica las credenciales contra el hash almacenado.
    4. El sistema genera un token JWT con validez de 24 horas y lo devuelve.
    5. El frontend almacena el token y redirige a la pantalla de inicio.
- **Flujo alternativo (credenciales incorrectas):** 3a. Las credenciales no coinciden. 3b. El sistema devuelve un error 401 con detalle: "Credenciales incorrectas".
- **Postcondición:** El usuario tiene un token JWT válido.
- **Nivel:** 1 (MVP)

CU-03: Cerrar sesión

- **Actor principal:** Usuario autenticado
- **Precondición:** El usuario tiene sesión iniciada (token JWT almacenado en el frontend).
- **Flujo principal:**
    1. El usuario pulsa "Cerrar sesión".
    2. El frontend elimina el token JWT almacenado.
    3. El frontend redirige a la pantalla de login.
- **Postcondición:** El usuario ya no tiene token JWT y no puede acceder a rutas protegidas.
- **Nivel:** 1 (MVP)

CU-04: Buscar película

- **Actor principal:** Visitante o Usuario autenticado
- **Actor secundario:** API de TMDB
- **Precondición:** Ninguna (accesible sin autenticación).
- **Flujo principal:**
    1. El usuario escribe un título en la barra de búsqueda.
    2. El frontend envía la consulta al backend.
    3. El backend llama a la API de TMDB con el término de búsqueda.
    4. TMDB devuelve una lista de películas que coinciden.
    5. El backend devuelve los resultados al frontend paginados.
    6. El frontend muestra las tarjetas de películas (póster, título, año).
- **Flujo alternativo (TMDB no responde):** 3a. La llamada a TMDB falla o tarda más de 5 segundos. 3b. El sistema devuelve un error claro: "No se pudieron obtener resultados. Inténtalo de nuevo."
- **Flujo alternativo (sin resultados):** 4a. TMDB no devuelve ningún resultado. 4b. El sistema muestra un mensaje: "No se encontraron películas para ese título."
- **Postcondición:** El usuario ve los resultados de búsqueda.
- **Nivel:** 1 (MVP)

CU-05: Ver detalle de película

- **Actor principal:** Visitante o Usuario autenticado
- **Actor secundario:** API de TMDB
- **Precondición:** El usuario o visitante ha buscado una película o está navegando la cartelera.
- **Flujo principal:**
    1. El usuario pulsa en una tarjeta de película.

    2. El frontend solicita al backend el detalle de la película por su tmdb_id.
    3. El backend consulta a TMDB los datos completos (sinopsis, reparto, géneros, etc.).
    4. Si el usuario está autenticado, el backend también consulta la base de datos local para obtener su estado con esa película (si la tiene como vista, su puntuación, si está en watchlist, si le ha dado me gusta). Si es un visitante, se omite este paso.
    5. El frontend muestra la información de la película. Si es usuario autenticado, los botones de acción muestran su estado actual. Si es visitante, los botones están deshabilitados o redirigen al login.
- **Flujo alternativo (visitante intenta interactuar):** 5a. Un visitante pulsa un botón de acción (vista, puntuación, etc.). 5b. El sistema redirige a la pantalla de login/registro.
- **Postcondición:** El usuario ve la ficha completa de la película.
- **Nivel:** 1 (MVP). En Nivel 2 se añaden las secciones de reseñas generales (visibles para visitantes) y reseñas de amigos (solo para usuarios autenticados).

CU-06: Marcar/desmarcar como vista

- **Actor principal:** Usuario autenticado. Precondición: El usuario está viendo el detalle de una película.
- **Flujo principal (marcar):**
    1. El usuario pulsa el botón "Vista".
    2. El frontend envía una petición POST al backend.
    3. El backend verifica que no existe ya un registro de Vista para ese usuario-película.
    4. Si la película no existe en la tabla Pelicula, se crea con los datos de caché (tmdb_id, título, póster, año).
    5. Se crea el registro de Vista. Si la película estaba en la watchlist del usuario, se elimina automáticamente.
    6. El frontend actualiza el botón para indicar que la película está marcada como vista.
- **Flujo principal (desmarcar):**
    1. La película ya está marcada como vista y el usuario no tiene entradas de diario, ni puntuación, ni me gusta para esa película. El botón de desmarcar está activo.
    2. El usuario pulsa el botón "Vista" de nuevo.
    3. El frontend envía una petición DELETE al backend.
    4. Se elimina el registro de Vista.
    5. El frontend actualiza el botón.
- **Flujo alternativo (no se puede desmarcar):**
    - 1a. La película está marcada como vista pero tiene datos asociados (entradas de diario, puntuación o me gusta).
    - 1b. El botón de desmarcar está deshabilitado. Se muestra un indicador visual explicando por qué y qué debe quitar antes de poder desmarcar (ej: 'Elimina tu puntuación primero', 'Quita el me gusta primero').
- **Postcondición**: Existe o se elimina un registro de Vista para ese usuario-película.
- **Nivel:** 1 (MVP)

CU-07: Puntuar película

- **Actor principal:** Usuario autenticado
- **Precondición:** El usuario está viendo el detalle de una película.
- **Flujo principal:**
    1. El usuario selecciona una puntuación del 1 al 10.
    2. El frontend envía una petición PUT al backend con la puntuación.

    3. El backend valida que la puntuación es un entero entre 1 y 10.
    4. Si existe un registro de Vista para ese usuario-película, se actualiza la puntuación.
    5. Si no existe Vista, se crea automáticamente con la puntuación (la película queda marcada como vista). Si no existe Vista y se crea automáticamente, y la película estaba en la watchlist, se elimina automáticamente.
    6. El frontend refleja la puntuación actualizada.
- **Flujo alternativo (puntuación inválida):** 3a. La puntuación no está entre 1 y 10 o no es un entero. 3b. El sistema devuelve un error 422 con detalle: "La puntuación debe ser un entero entre 1 y 10."
- **Postcondición:** La puntuación del usuario para esa película está actualizada en Vista.
- **Nivel:** 1 (MVP)

CU-08: Registrar en diario

- **Actor principal:** Usuario autenticado
- **Precondición:** El usuario está viendo el detalle de una película.
- **Flujo principal:**
    1. El usuario pulsa "Añadir al diario".
    2. Se abre un formulario con fecha de visionado (por defecto hoy) y campo de reseña (opcional).
    3. El usuario rellena los datos y pulsa guardar.
    4. El backend crea una nueva Entrada_Diario.
    5. Si la película no estaba marcada como vista, se crea automáticamente un registro de Vista. Si la película estaba en la watchlist, se elimina automáticamente.
    6. El frontend muestra confirmación.
- **Postcondición:** Existe una nueva entrada en el diario. La película está marcada como vista.
- **Nivel:** 1 (MVP)

CU-09: Seguir a un usuario

- **Actor principal:** Usuario autenticado
- **Precondición:** El usuario está viendo el perfil de otro usuario.
- **Flujo principal:**
    1. El usuario pulsa "Seguir".
    2. El frontend envía una petición POST al backend.
    3. El backend crea un registro en la tabla Seguidor (id_seguidor = usuario actual, id_seguido = otro usuario).
    4. El frontend actualiza el botón a "Siguiendo".
- **Flujo alternativo (dejar de seguir):** 1a. El usuario ya sigue al otro. 1b. Pulsa "Siguiendo" / "Dejar de seguir". 1c. Se elimina el registro de Seguidor.
- **Postcondición:** El usuario sigue (o deja de seguir) al otro usuario.
- **Nivel:** 2 (Social)

CU-10: Ver reseñas de amigos en una película

- **Actor principal:** Usuario autenticado
- **Precondición:** El usuario está viendo el detalle de una película.
- **Flujo principal:**
    1. El detalle de la película muestra una sección "Tus amigos".

    2. El backend consulta qué usuarios seguidos por el usuario actual tienen registros (Vista o Entrada_Diario) para esa película.
    3. Se muestra una tarjeta por cada amigo: avatar, nombre, puntuación si la tiene, y un fragmento de su última reseña si la tiene.
    4. Si el amigo tiene reseña, el usuario puede pulsar para ver todas sus entradas de diario para esa película.
    5. Si el amigo solo tiene Vista sin reseña, se muestra su puntuación pero no hay enlace a más detalle.
- **Postcondición:** El usuario ve la actividad de sus amigos para esa película.
- **Nivel:** 2 (Social)

CU-11: Comentar en una reseña

- **Actor principal:** Usuario autenticado
- **Precondición:** El usuario está viendo una entrada de diario de cualquier usuario.
- **Flujo principal:**
    1. El usuario escribe un comentario en el campo de texto debajo de la reseña.
    2. Pulsa enviar.
    3. El backend crea un registro de ComentarioResena asociado a esa entrada de diario.
    4. El comentario aparece debajo de la reseña con el avatar y username del autor y la fecha.
- **Postcondición:** Existe un nuevo comentario asociado a la entrada de diario.
- **Nivel:** 2 (Social)

CU-12: Crear y gestionar una lista personalizada

- **Actor principal:** Usuario autenticado
- **Precondición:** El usuario está autenticado.
- **Flujo principal (crear lista):**
    1. El usuario va a su perfil y pulsa "Crear nueva lista".
    2. Introduce nombre, descripción (opcional) y elige si es pública o privada.
    3. El backend crea un registro en la tabla Lista.
    4. La lista aparece en su perfil (visible para todos si es pública, solo para él si es privada).
- **Flujo principal (añadir película a lista):**
    1. El usuario está en el detalle de una película y pulsa "Añadir a lista".
    2. Se muestra un selector con sus listas disponibles.
    3. Selecciona una lista. El backend crea un registro en ListaPelicula.
- **Flujo principal (ver lista ajena):**
    1. Un visitante o usuario visita el perfil de otro usuario.
    2. Ve sus listas públicas. Pulsa en una.
    3. Se muestran las películas de la lista con paginación.
- **Flujo alternativo (lista privada):** 1a. Un visitante o usuario intenta acceder a una lista privada de otro usuario. 1b. El sistema devuelve un error 403: "Esta lista es privada."
- **Postcondición:** La lista existe y contiene las películas añadidas.
- **Nivel:** 2 (Social)

CU-13: Reclamar XP de un logro

- **Actor principal:** Usuario autenticado

- **Precondición:** El usuario tiene al menos un logro desbloqueado sin reclamar.
- **Flujo principal:**
    1. El usuario va a la sección de logros en su perfil.
    2. Ve un logro con el botón "Reclamar XP".
    3. Pulsa el botón.
    4. El backend suma la XP del logro al xp_total del usuario, recalcula el nivel y marca el logro como reclamado.
    5. El frontend actualiza la XP, el nivel y el estado del logro.
- **Postcondición:** El logro está marcado como reclamado, la XP del usuario ha aumentado y su nivel puede haber subido.
- **Nivel:** 3 (Gamificación)

7. Hoja de Ruta del Producto

El desarrollo de Bonobo se divide en tres niveles incrementales. Cada nivel se construye sobre el anterior y añade una capa de funcionalidad. El proyecto es funcional y aporta valor al usuario desde el Nivel 1.

Nivel 1 — MVP (Producto Mínimo Viable)

Tema: La experiencia individual completa.

Descripción: Cualquier persona puede visitar Bonobo y navegar: ver la cartelera, buscar películas, consultar detalles y ver perfiles de otros usuarios sin necesidad de cuenta. Para interactuar (marcar como vista, puntuar, escribir reseñas, etc.) es necesario registrarse. El usuario registrado puede buscar películas, interactuar con ellas de todas las formas posibles (vista, puntuación, diario, me gusta, watchlist, favoritas) y gestionar su perfil completo. La app funciona como un diario cinematográfico personal completo.

Incluye: Navegación pública sin cuenta (cartelera, estrenos, búsqueda, detalle, perfiles), autenticación (registro, login, logout), marcar como vista, puntuar (1-10 enteros), registrar en diario con reseña, me gusta, watchlist, favoritas, perfil de usuario (ver, editar, foto, bio), ver perfiles de otros usuarios, búsqueda de usuarios, paginación en todos los listados, manejo de errores consistente.

Criterio de completado: Álex puede abrir Bonobo, registrarse, buscar "Dune", marcarla como vista, darle un 8, escribir una reseña en el diario, añadir otra película a su watchlist, y ver todo esto reflejado en su perfil. Si esto funciona, el Nivel 1 está completo.

Nivel 2 — Social + Listas

Tema: Conectar con otros usuarios y organizar películas.

Descripción: El usuario puede seguir a otros usuarios y ver su actividad. En el detalle de cada película aparecen las reseñas de los amigos y las reseñas generales de la comunidad. Se puede comentar y dar like a las reseñas. Además, el usuario puede crear listas personalizadas de películas (públicas o privadas) para organizar su colección como quiera. Bonobo pasa de ser un diario personal a una red social cinematográfica.

Incluye: Sistema de seguidores (seguir/dejar de seguir), sección de reseñas de amigos en el detalle de película, sección de reseñas generales en el detalle de película, comentarios en reseñas (sin respuestas anidadas), likes en reseñas, contador de seguidores/seguidos en perfil, lista de seguidores/seguidos, listas personalizadas de películas (crear, editar, eliminar, pública/privada), ver listas públicas de otros usuarios.

Prerequisitos: Nivel 1 completado.

Criterio de completado: Álex sigue a Mario, abre la ficha de "Oppenheimer" y ve la reseña de Mario en la sección de amigos. Puede darle like y escribir un comentario. En la sección general ve reseñas de gente que no sigue. Además, Álex crea una lista "Ciencia ficción imprescindible" pública con 4 películas, y Mario puede verla desde el perfil de Álex.

Nivel 3 — Gamificación

Tema: Progresión y competición.

Descripción: El usuario desbloquea logros al completar ciertas acciones, reclama puntos de experiencia (XP) y sube de nivel. Puede ver un ranking de nivel entre las personas que sigue. Esta capa convierte el uso habitual de la app en un juego y es el diferenciador principal de Bonobo frente a otras plataformas.

Incluye: Catálogo de logros con condiciones de desbloqueo, desbloqueo automático de logros, reclamar XP de logros, cálculo de nivel (XP_total / 100), mostrar nivel y XP en perfil, ranking de nivel entre amigos, listado de logros disponibles y progreso.

Prerequisitos: Nivel 1 y Nivel 2 completados (los logros pueden referirse a acciones sociales como "conseguir 10 seguidores").

Criterio de completado: Álex registra su décima película, se le desbloquea el logro "Cinéfilo en progreso", reclama 50 XP, sube al nivel 3 y ve que ha adelantado a Bladi en el ranking.

Resumen visual

NIVEL 1 (MVP)	NIVEL 2 (Social + Listas)	NIVEL 3 (Gamificación)
Auth	Seguidores	Logros
Buscar películas	Reseñas de amigos	XP y niveles
Vista + Puntuación	Reseñas generales	Ranking de amigos
Diario + Reseña	Comentarios en reseñas	
Me gusta	Likes en reseñas	
Watchlist + Favoritas	Seguidores/seguidos	
Perfil completo	Listas personalizadas	

"Diario personal" → "Red social de cine" → "Juego de cine"

8. Arquitectura del Sistema

8.1. Descripción General

Bonobo utiliza una arquitectura cliente-servidor con separación total entre frontend y backend. Los tres componentes principales son:

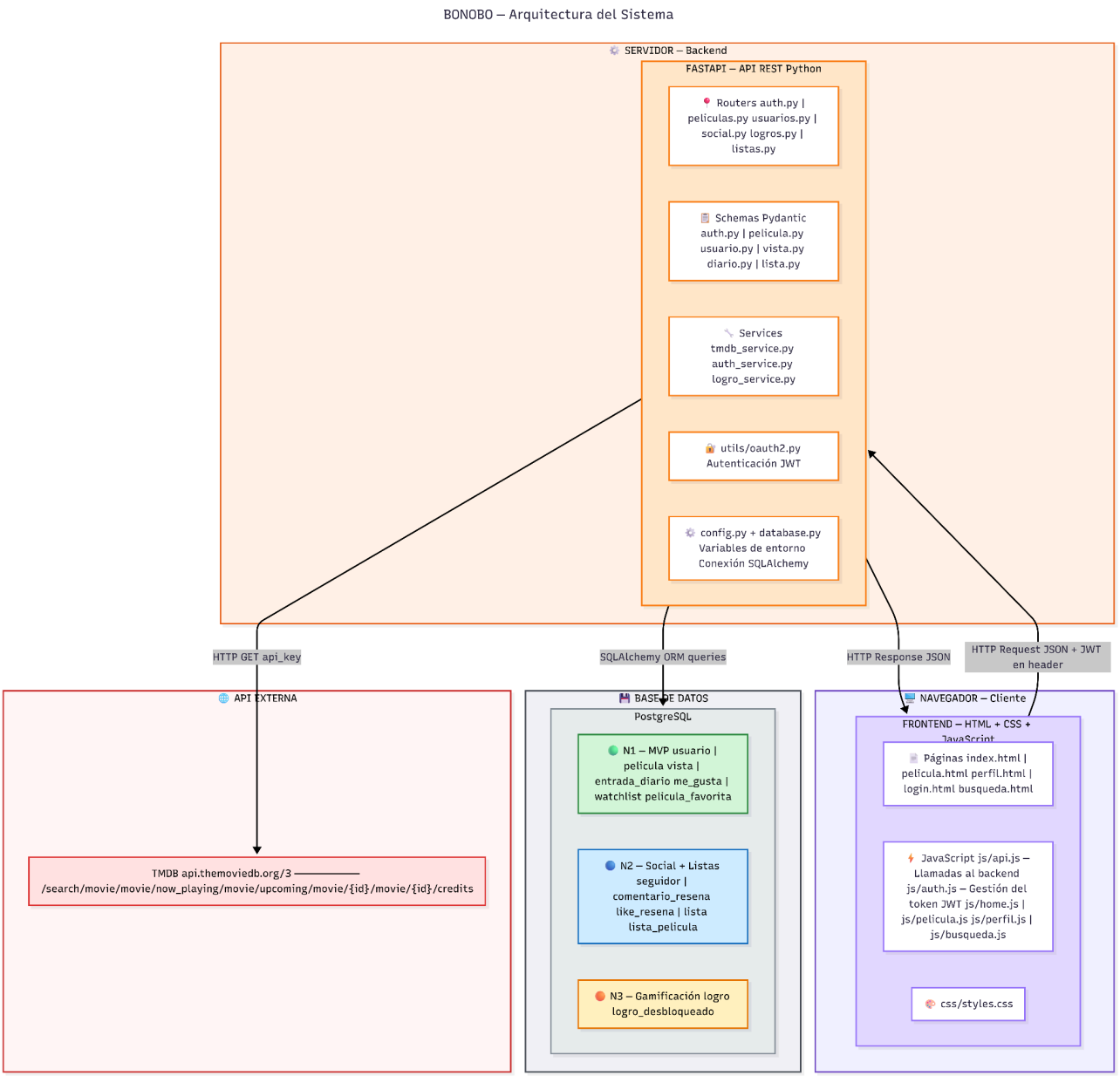
Frontend (cliente): Aplicación web construida con HTML, CSS y JavaScript puro. Se ejecuta en el navegador del usuario. Se comunica con el backend exclusivamente mediante peticiones HTTP enviando y recibiendo JSON. Almacena el token JWT en el lado del cliente para mantener la sesión.

Backend (servidor): API REST construida con Python y FastAPI. Recibe peticiones del frontend, ejecuta la lógica de negocio, interactúa con la base de datos mediante SQLAlchemy y consume la API de TMDB cuando necesita datos de películas. Gestiona la autenticación mediante JWT. Todas las respuestas son JSON.

Base de datos: PostgreSQL. Almacena todos los datos de la aplicación: usuarios, vistas, diario, interacciones sociales, logros, etc. Los datos de películas se almacenan solo como caché mínimo (tmdb_id, título, póster, año); los datos completos se piden a TMDB en tiempo real.

API externa (TMDB): Servicio de terceros que proporciona los datos de películas. El backend consulta TMDB para búsquedas, cartelera, estrenos y detalle de películas. La comunicación es del backend a TMDB directamente; el frontend nunca habla con TMDB.

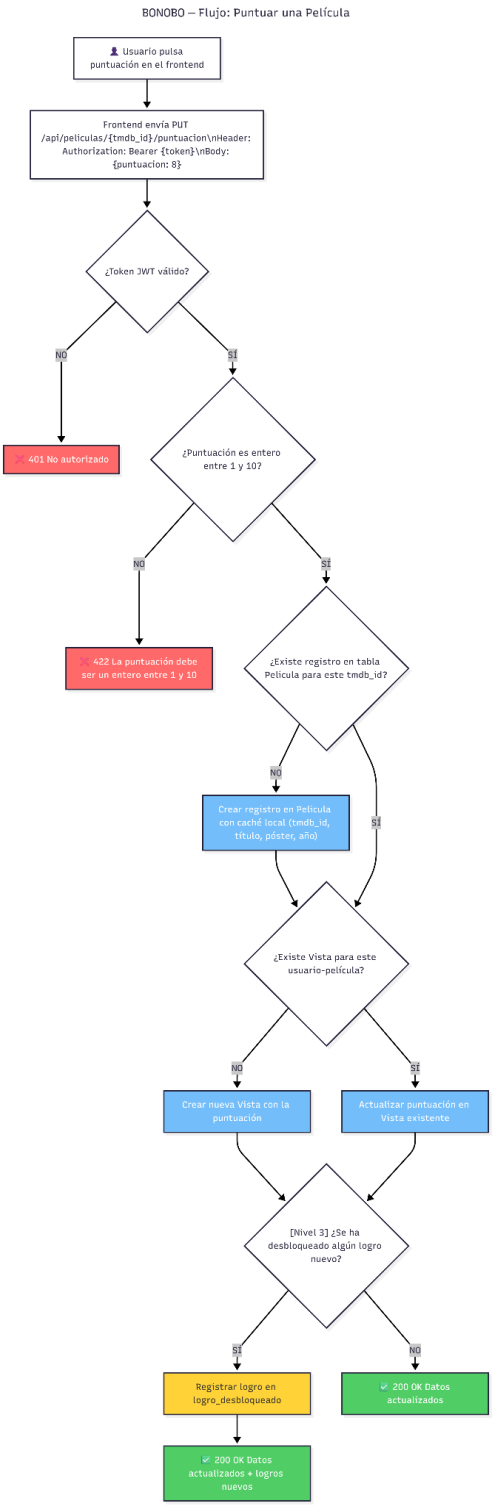
8.2. Diagrama de Arquitectura



8.3. Flujo de una Petición Típica

Ejemplo: un visitante busca "Dune" (endpoint público)

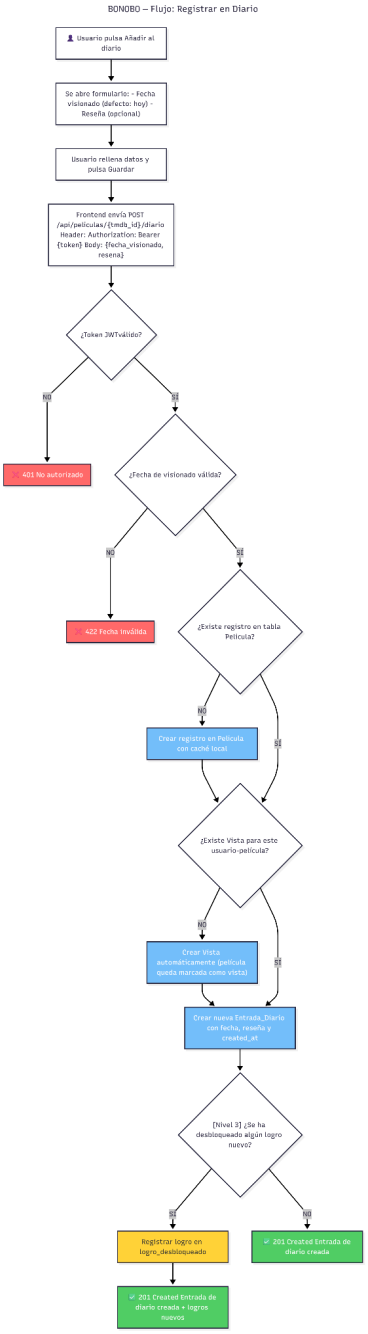
1. El visitante (o usuario) escribe "Dune" en el buscador del frontend.
2. El frontend (js/api.js) hace una petición GET a `http://backend/api/peliculas/buscar?q=Dune&skip=0&limit=20`. No se requiere token JWT.
3. El backend (routers/peliculas.py) recibe la petición.
4. El backend (services/tmdb_service.py) llama a la API de TMDB: `https://api.themoviedb.org/3/search/movie?query=Dune&api_key={key}`.
5. TMDB devuelve los resultados en JSON.
6. El backend formatea los resultados y los devuelve al frontend como JSON paginado.
7. El frontend pinta las tarjetas de películas con los datos recibidos.



Ejemplo: un usuario autenticado puntúa una película (endpoint protegido 🔒)

1. El usuario selecciona un 8 en la ficha de "Dune".
2. El frontend hace PUT a `/api/peliculas/{tmdb_id}/puntuacion` con body `{"puntuacion": 8}` y header JWT.
3. El backend verifica el JWT, valida que la puntuación es un entero entre 1 y 10.
4. El backend busca si existe un registro en la tabla Pelicula para ese tmdb_id. Si no existe, lo crea con datos de caché.
5. El backend busca si existe una Vista para ese usuario-película.
    - Si existe: actualiza el campo puntuacion.

    - Si no existe: crea una nueva Vista con la puntuación.
6. (Nivel 3) El backend comprueba si se ha desbloqueado algún logro nuevo.
7. Devuelve 200 OK con los datos actualizados.



9. Modelo de Datos

9.1. Modelo Conceptual

El modelo conceptual describe las entidades del sistema y sus relaciones sin entrar en detalles técnicos como tipos de datos o claves primarias.

Entidades y su significado:

Usuario — La persona que usa la aplicación. Tiene datos de perfil y, en el Nivel 3, datos de progresión.

Película — Una película concreta, identificada por su ID de TMDB. Almacena un caché mínimo (título, póster, año) para no depender de TMDB al mostrar listados.

Vista — El hecho de que un usuario ha visto una película. Aquí vive la puntuación (del 1 al 10, entera). Solo puede existir una Vista por usuario-película.

Entrada de Diario — Un registro detallado de cuándo el usuario vio una película. Incluye fecha de visionado y reseña opcional. Un usuario puede tener muchas entradas de diario para la misma película (la vio varias veces). Esta es también la "reseña" que ven otros usuarios en el Nivel 2.

Me Gusta — Marca que un usuario considera una película como especial. Un solo me gusta por usuario-película.

Watchlist — Películas que el usuario quiere ver en el futuro. Una entrada por usuario-película.

Favorita — Películas que el usuario elige para destacar en la cabecera de su perfil. Se configuran desde la edición del perfil, no desde la ficha de la película. Máximo 4 por usuario. Incluyen un campo de orden (posición 1 a 4) para controlar cómo se muestran.

Seguidor (Nivel 2) — Relación de seguimiento entre dos usuarios. Si A sigue a B, A es seguidor de B y B es seguido de A. No requiere aceptación. Si ambos se siguen mutuamente, se consideran "amigos".

Comentario de Reseña (Nivel 2) — Un comentario de texto plano que un usuario escribe en una entrada de diario de cualquier usuario. Sin respuestas anidadas. Una entrada de diario puede tener muchos comentarios.

Like de Reseña (Nivel 2) — Un like que un usuario da a una entrada de diario de cualquier usuario. Un solo like por usuario-entrada.

Lista (Nivel 2) — Una colección personalizada de películas creada por un usuario. Tiene nombre, descripción opcional y puede ser pública (visible para todos) o privada (solo visible para su creador). Un usuario puede crear muchas listas.

Lista-Película (Nivel 2) — La relación entre una lista y una película. Una lista puede contener muchas películas, y una película puede estar en muchas listas. Solo una entrada por lista-película.

Logro (Nivel 3) — La definición de un logro (nombre, descripción, XP que otorga). Existe independientemente de los usuarios. Ejemplos: "Primera reseña", "10 películas vistas", "Maratonista (10 pelis en un mes)".

Logro Desbloqueado (Nivel 3) — La relación entre un usuario y un logro que ha completado. Incluye si el usuario ya reclamó la XP.

9.2. Relaciones entre Entidades

Nivel 1 (MVP):

- Un Usuario puede tener muchas Vistas, pero solo una por Película.

- Un Usuario puede tener muchas Entradas de Diario, incluso varias para la misma Película.
- Un Usuario puede tener muchos Me Gusta, pero solo uno por Película.
- Un Usuario puede tener muchas Películas en su Watchlist, pero solo una entrada por Película.
- Un Usuario puede tener como máximo 4 Favoritas, una por Película.
- Una Película puede ser vista, puntuada, reseñada, gustada y favoritada por muchos Usuarios.

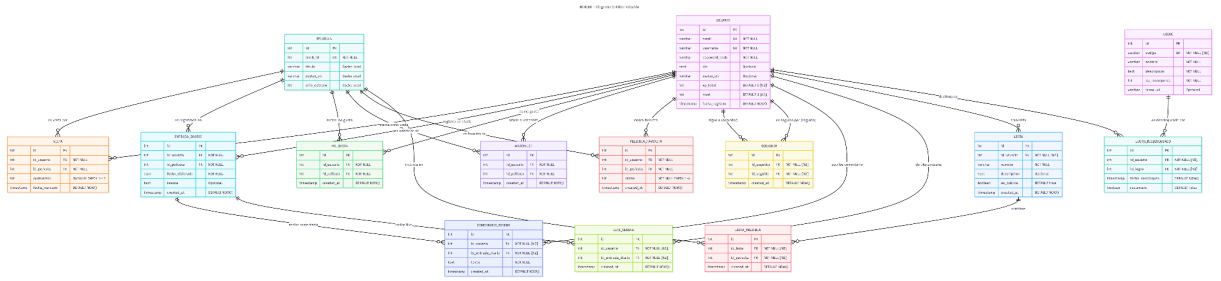
Nivel 2 (Social + Listas):

- Un Usuario puede seguir a muchos Usuarios y ser seguido por muchos Usuarios.
- Una Entrada de Diario puede tener muchos Comentarios de distintos Usuarios.
- Una Entrada de Diario puede tener muchos Likes de distintos Usuarios, pero solo un Like por Usuario.
- Un Usuario puede crear muchas Listas.
- Una Lista puede contener muchas Películas, y una Película puede estar en muchas Listas (relación N:M a través de Lista-Película).
- Solo una entrada por Lista-Película (no puedes añadir la misma película dos veces a la misma lista).

Nivel 3 (Gamificación):

- Un Usuario puede desbloquear muchos Logros, pero cada Logro solo se desbloquea una vez por Usuario.
- Un Logro puede ser desbloqueado por muchos Usuarios.

9.3. Diagrama Entidad-Relación



9.4. Tablas de la Base de Datos

Nivel 1 — MVP

Tabla: usuario

Campo	Tipo	Restricciones
id	SERIAL	PK
email	VARCHAR(255)	UNIQUE, NOT NULL
username	VARCHAR(50)	UNIQUE, NOT NULL

password_hash	VARCHAR(255)	NOT NULL
bio	TEXT	Opcional
avatar_url	VARCHAR(500)	Opcional
xp_total	INTEGER	DEFAULT 0 (usado en Nivel 3)
nivel	INTEGER	DEFAULT 1 (usado en Nivel 3)
fecha_registro	TIMESTAMP	DEFAULT NOW()

Tabla: pelicula

Campo	Tipo	Restricciones
id	SERIAL	PK
tmdb_id	INTEGER	UNIQUE, NOT NULL
titulo	VARCHAR(255)	Caché local
poster_url	VARCHAR(500)	Caché local
anio_estreno	INTEGER	Caché local

Tabla: vista

Campo	Tipo	Restricciones
id	SERIAL	PK
id_usuario	INTEGER	FK → usuario(id), NOT NULL
id_pelicula	INTEGER	FK → pelicula(id), NOT NULL
puntuacion	INTEGER	Opcional, CHECK (1 <= puntuacion <= 10)
fecha_marcado	TIMESTAMP	DEFAULT NOW()
		UNIQUE(id_usuario, id_pelicula)

Tabla: entrada_diario

Campo	Tipo	Restricciones
id	SERIAL	PK

id_usuario	INTEGER	FK → usuario(id), NOT NULL
id_pelicula	INTEGER	FK → pelicula(id), NOT NULL
fecha_visionado	DATE	NOT NULL
resena	TEXT	Opcional
created_at	TIMESTAMP	DEFAULT NOW()

Sin restricción UNIQUE en (id_usuario, id_pelicula): un usuario puede tener múltiples entradas de diario para la misma película.

Tabla: me_gusta

Campo	Tipo	Restricciones
id	SERIAL	PK
id_usuario	INTEGER	FK → usuario(id), NOT NULL
id_pelicula	INTEGER	FK → pelicula(id), NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()
		UNIQUE(id_usuario, id_pelicula)

Tabla: watchlist

Campo	Tipo	Restricciones
id	SERIAL	PK
id_usuario	INTEGER	FK → usuario(id), NOT NULL
id_pelicula	INTEGER	FK → pelicula(id), NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()
		UNIQUE(id_usuario, id_pelicula)

Tabla: pelicula_favorita

Campo	Tipo	Restricciones
id	SERIAL	PK
id_usuario	INTEGER	FK → usuario(id), NOT NULL
id_pelicula	INTEGER	FK → pelicula(id), NOT NULL
orden	INTEGER	NOT NULL, CHECK (1 <= orden <= 4)

created_at	TIMESTAMP	DEFAULT NOW()
		UNIQUE(id_usuario, id_pelicula)
		UNIQUE(id_usuario, orden)

Máximo 4 registros por usuario (validado en el backend). El campo orden (1-4) determina la posición en el perfil. UNIQUE(id_usuario, orden) garantiza que no haya dos películas en la misma posición.

Nivel 2 — Social

Tabla: seguidor

Campo	Tipo	Restricciones
id	SERIAL	PK
id_seguidor	INTEGER	FK → usuario(id), NOT NULL
id_seguido	INTEGER	FK → usuario(id), NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()
		UNIQUE(id_seguidor, id_seguido)
		CHECK(id_seguidor != id_seguido)

Tabla: comentario_resena

Campo	Tipo	Restricciones
id	SERIAL	PK
id_usuario	INTEGER	FK → usuario(id), NOT NULL
id_entrada_diario	INTEGER	FK → entrada_diario(id), NOT NULL
texto	TEXT	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Tabla: like_resena

Campo	Tipo	Restricciones
id	SERIAL	PK
id_usuario	INTEGER	FK → usuario(id), NOT NULL
id_entrada_diario	INTEGER	FK → entrada_diario(id), NOT NULL

created_at	TIMESTAMP	DEFAULT NOW()
		UNIQUE(id_usuario, id_entrada_diario)

Tabla: lista

Campo	Tipo	Restricciones
id	SERIAL	PK
id_usuario	INTEGER	FK → usuario(id), NOT NULL
nombre	VARCHAR(100)	NOT NULL
descripcion	TEXT	Opcional
es_publica	BOOLEAN	DEFAULT true
created_at	TIMESTAMP	DEFAULT NOW()

Tabla: lista_pelicula

Campo	Tipo	Restricciones
id	SERIAL	PK
id_lista	INTEGER	FK → lista(id), NOT NULL
id_pelicula	INTEGER	FK → pelicula(id), NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()
		UNIQUE(id_lista, id_pelicula)

Nivel 3 — Gamificación

Tabla: logro

Campo	Tipo	Restricciones
id	SERIAL	PK
codigo	VARCHAR(50)	UNIQUE, NOT NULL
nombre	VARCHAR(100)	NOT NULL
descripcion	TEXT	NOT NULL
xp_recompensa	INTEGER	NOT NULL
icono_url	VARCHAR(500)	Opcional

Tabla: logro_desbloqueado

Campo	Tipo	Restricciones
id	SERIAL	PK
id_usuario	INTEGER	FK → usuario(id), NOT NULL
id_logro	INTEGER	FK → logro(id), NOT NULL
fecha_desbloqueo	TIMESTAMP	DEFAULT NOW()
reclamado	BOOLEAN	DEFAULT false
		UNIQUE(id_usuario, id_logro)

10. Diseño de la API

Todos los endpoints devuelven JSON. El token JWT se envía en el header Authorization: Bearer {token}.

Tres tipos de autenticación en los endpoints:

- **Si Requiere autenticación:** El endpoint exige un token JWT válido. Sin token o con token inválido, devuelve 401.
- **No Requiere autenticación(público):** El endpoint funciona sin token. Cualquier visitante puede acceder.
- ***No (pero enriquecido con token):** El endpoint funciona sin token, pero si el usuario envía un token JWT válido, la respuesta incluye información adicional personalizada. Por ejemplo, GET /api/peliculas/{tmdb_id} devuelve a todos los datos de TMDB, pero si el usuario está autenticado, el backend añade su estado con la película (vista, puntuación, watchlist, etc.).

Autenticación

Método	Endpoint	Descripción
POST	/api/auth/registro	Crear cuenta (body: email, username, password). Devuelve los datos del usuario creado.
POST	/api/auth/login	Iniciar sesión (body: email, password). Devuelve el token JWT.

Películas

Método	Endpoint	Descripción	Nivel	Auth

GET	/api/peliculas/buscar?q={titulo}&skip=0&limit=20	Buscar películas por título en TMDB.	N1	No
GET	/api/peliculas/cartelera?skip=0&limit=20	Películas en cartelera actual (TMDB).	N1	No
GET	/api/peliculas/estrenos?skip=0&limit=20	Próximos estrenos (TMDB).	N1	No
GET	/api/peliculas/{tmdb_id}	Detalle completo de una película (TMDB). Si se envía JWT, incluye además el estado del usuario (vista, puntuación, me gusta, watchlist).	N1	No*
GET	/api/peliculas/{tmdb_id}/resenas/amigos?skip=0&limit=20	Reseñas de usuarios que sigo para esta película.	N2	Si
GET	/api/peliculas/{tmdb_id}/resenas?skip=0&limit=20	Reseñas generales de todos los usuarios para esta película.	N2	No

Acciones sobre Películas

Método	Endpoint	Descripción	Nivel	Auth
POST	/api/peliculas/{tmdb_id}/vista	Marcar como vista.	N1	Si
DELETE	/api/peliculas/{tmdb_id}/vista	Desmarcar como vista.Solo si no tiene entradas de diario, puntuación ni me gusta. Devuelve 409 como protección si se intenta sin cumplir las condiciones.	N1	Si
PUT	/api/peliculas/{tmdb_id}/puntuacion	Puntuar (body: {"puntuacion": 8}). Crea Vista si no existe.	N1	Si
POST	/api/peliculas/{tmdb_id}/diario	Crear entrada de diario (body: {"fecha_visionado": "2024-03-15", "resena": "..."}).	N1	Si
POST	/api/peliculas/{tmdb_id}/me-gusta	Dar me gusta.	N1	Si
DELETE	/api/peliculas/{tmdb_id}/me-gusta	Quitar me gusta.	N1	Si

POST	/api/peliculas/{tmdb_id}/watchlist	Añadir a watchlist.	N1	Si
DELETE	/api/peliculas/{tmdb_id}/watchlist	Quitar de watchlist.	N1	Si
DELETE	/api/peliculas/{tmdb_id}/puntuacion	Eliminar puntuación (vuelve a null). La Vista se mantiene.	N1	Si
PUT	/api/diario/{id_entrada}	Editar entrada de diario propia (body: {"fecha_visionado": "...", "resena": "..."}).	N1	Si

Perfil y Usuarios

Método	Endpoint	Descripción	Nivel	Auth
GET	/api/usuarios/me	Mi perfil completo (datos, contadores).	N1	Si
PUT	/api/usuarios/me	Editar mi perfil (body: {"bio": "...", "avatar_url": "..."}).	N1	Si
PUT	/api/usuarios/me/favoritas	Configurar películas favoritas (body: {"tmdb_ids": [123, 456, 789, 101]}). Máximo 4. Reemplaza las anteriores.	N1	Si
GET	/api/usuarios/buscar?q={username}&skip=0&limit=20	Buscar usuarios por username.	N1	No
GET	/api/usuarios/{id}	Ver perfil de otro usuario (incluye sus favoritas).	N1	No
GET	/api/usuarios/{id}/vistas?skip=0&limit=20	Películas vistas de un usuario.	N1	No
GET	/api/usuarios/{id}/diario?skip=0&limit=20	Diario de un usuario.	N1	No
GET	/api/usuarios/{id}/watchlist?skip=0&limit=20	Watchlist de un usuario.	N1	No
GET	/api/usuarios/{id}/favoritas	Las 4 (máximo) películas favoritas de un usuario.	N1	No

Social

Método	Endpoint	Descripción	Nivel	Auth

POST	/api/usuarios/{id}/seguir	Seguir a un usuario.	N2	Si
DELETE	/api/usuarios/{id}/seguir	Dejar de seguir a un usuario.	N2	Si
GET	/api/usuarios/{id}/seguidores?skip=0&limit=20	Lista de seguidores de un usuario.	N2	No
GET	/api/usuarios/{id}/seguidos?skip=0&limit=20	Lista de seguidos de un usuario.	N2	No
POST	/api/diario/{id_entrada}/comentarios	Comentar en una entrada de diario (body: {"texto": "..."}).	N2	Si
GET	/api/diario/{id_entrada}/comentarios?skip=0&limit=20	Ver comentarios de una entrada de diario.	N2	No
DELETE	/api/comentarios/{id_comentario}	Eliminar un comentario propio.	N2	Si
POST	/api/diario/{id_entrada}/like	Dar like a una entrada de diario.	N2	Si
DELETE	/api/diario/{id_entrada}/like	Quitar like de una entrada de diario.	N2	Si

Diario Expandido

Método	Endpoint	Descripción	Nivel	Auth
GET	/api/usuarios/{id}/diario/pelicula/{tmdb_id}	Todas las entradas de diario de un usuario para una película concreta.	N2	No

Listas Personalizadas

Método	Endpoint	Descripción	Nivel	Auth
POST	/api/listas	Crear lista (body: {"nombre": "...", "descripcion": "...", "es_publica": true}).	N2	Si
GET	/api/usuarios/{id}/listas	Ver listas de un usuario (públicas para todos, públicas + privadas si es tu propio perfil).	N2	No*

GET	/api/listas/{id_lista}	Ver detalle de una lista (nombre, descripción, películas con paginación). Devuelve 403 si es privada y no eres el dueño.	N2	No*
PUT	/api/listas/{id_lista}	Editar lista propia (nombre, descripción, es_publica).	N2	Si
DELETE	/api/listas/{id_lista}	Eliminar lista propia.	N2	Si
POST	/api/listas/{id_lista}/peliculas/{tmdb_id}	Añadir película a lista propia.	N2	Si
DELETE	/api/listas/{id_lista}/peliculas/{tmdb_id}	Quitar película de lista propia.	N2	Si

*Los endpoints GET de listas son públicos pero con lógica condicional: si el usuario está autenticado y es el dueño, ve también sus listas privadas. Si no, solo ve las públicas.

Gamificación

Método	Endpoint	Descripción	Nivel	Auth
GET	/api/logros	Lista de todos los logros disponibles con estado (desbloqueado/no, reclamado/no).	N3	Si
GET	/api/usuarios/me/logros	Mis logros desbloqueados.	N3	Si
POST	/api/usuarios/me/logros/{id_logro}/reclamar	Reclamar XP de un logro desbloqueado.	N3	Si
GET	/api/usuarios/me/ranking?skip=0&limit=20	Ranking de nivel entre usuarios que sigo.	N3	Si

11. Paquete Tecnológico

Componente	Tecnología	Justificación
Backend	Python + FastAPI	Framework moderno con documentación automática Swagger, validación integrada con Pydantic y excelente rendimiento. Gran cantidad de recursos de aprendizaje disponibles.

Base de datos	PostgreSQL	Base de datos relacional robusta y gratuita. Excelente para las múltiples relaciones del modelo de Bonobo (vistas, diario, seguidores, logros).
ORM	SQLAlchemy	El ORM más utilizado en Python. Permite interactuar con PostgreSQL sin escribir SQL directamente y facilita las migraciones de esquema.
Autenticación	JWT (JSON Web Tokens)	Estándar de la industria para APIs REST. Sin estado (stateless), lo que simplifica el backend al no requerir sesiones en el servidor. Token con validez de 24 horas.
Frontend	HTML + CSS + JavaScript	Tecnologías web estándar sin dependencias de frameworks. Máxima compatibilidad, abundante documentación, y familiaridad universal. No requiere compilación ni herramientas de build.
API externa	TMDB (The Movie Database)	API gratuita y la más completa disponible para datos de películas. Ofrece búsqueda, cartelera, estrenos, detalle, reparto y pósters. Bien documentada.
Control de versiones	Git + GitHub	Estándar de la industria para desarrollo colaborativo. Permite trabajo en paralelo entre backend y frontend. GitHub ofrece gestión de issues y pull requests.
Despliegue	VM Ubuntu (Oracle Cloud / AWS)	Máximo control sobre el entorno de producción. Formativo para aprender administración de servidores. Capa gratuita disponible en ambos proveedores.

Equipo

Miembro	Rol
Valentín	Backend
Brian	Backend
Mario	Frontend
Bladi	Frontend